Layer 8

Cores, Threads & Video Decoding

Where concurrency meets real hardware

Concurrency vs. Parallelism

Petflix Box: 16-Core CPU

Core 1	Core 2	Core 3	Core 4
Core 5	Core 6	Core 7	Core 8
Core 9	Core 10	Core 11	Core 12
Core 13	Core 14	Core 15	Core 16

Concurrency

1,000 threads managed simultaneously. The system juggles many tasks, switching between them rapidly (context switching).

Parallelism

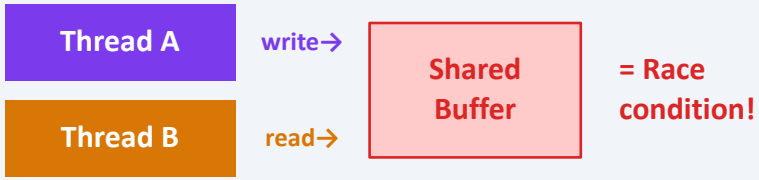
Only 16 things happen at the exact same instant — one per core. Multi-core CPUs make true parallelism real.

Thread Queue: 1,000 streams

16 running • 984 waiting

Race Conditions, Locks & Real-Time Decoding

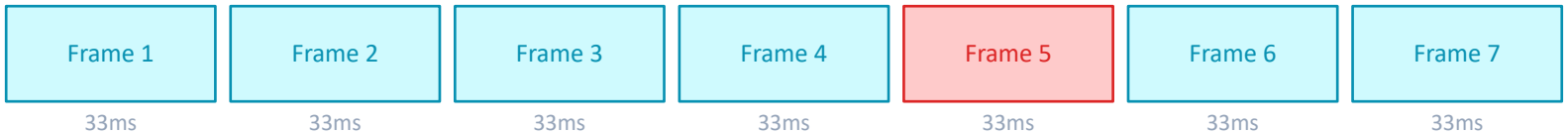
Shared Memory: The Danger



Solution: Locks (mutual exclusion)

Deadlock

Real-Time Decoding on Fatima's Phone



30 fps = 33ms per frame. Miss the deadline = visible stutter.

Hardware decoders handle the heavy lifting.

The Full Stack in One Picture

Layer	Concurrency Problem
DNS Routing	Global load balancing
API Gateway	10K concurrent connections
Microservices	Billions of internal calls/min
Cache & DB	400M cache ops/sec
CDN Box	1000s streams/box
Pods	Write serialization
VMs	Bin-packing containers
Cores	Race conditions, deadlines

Concurrency is not one problem.

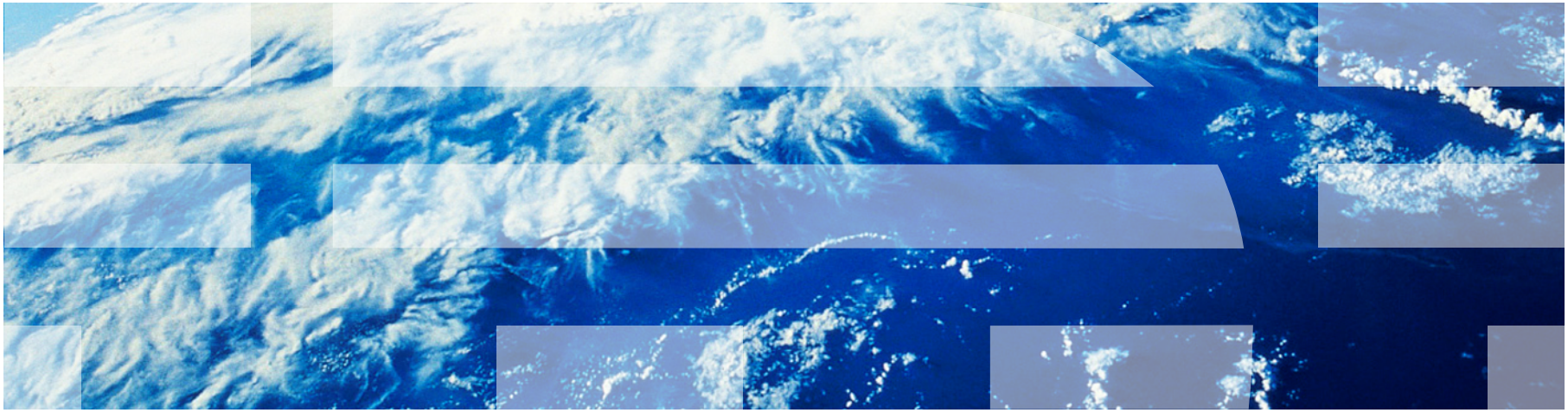
It's a different problem at every layer.

Our users don’t, and should not have to know any of this.

Borrowed from Shiva Shivakumar

Computer Systems for Data Science
Topic 3

Transactions and OLTP Databases



A note on OLTP vs. OLAP

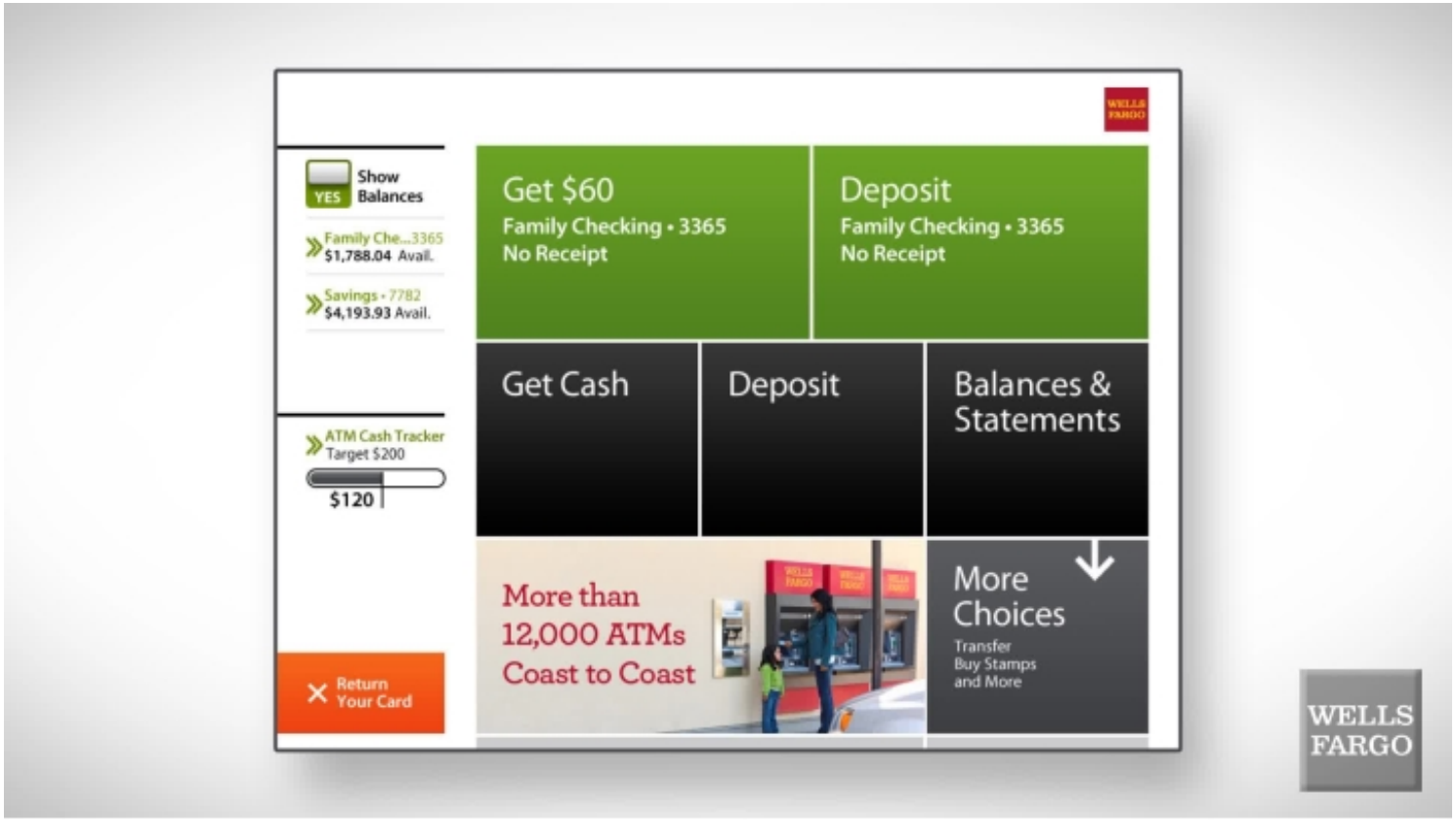
- There are broadly two types of databases in the world: OLTP and OLAP
- OLTP
  - Online Transaction Processing database
  - Supports reading and writing/updating data in real time
  - Provides **transactions** abstraction (usually, but not always!)
  - Usually smaller queries
  - Sometimes relational, sometimes not
  - Use cases
    - Financial transactions
    - Keep up-to-date progress in a game
    - Up-to-date user settings in a social media site
    - Google docs
  - Examples: MySQL, Postgres, Redis, Cassandra, DynamoDB, Aurora/RDS, …

A note on OLTP vs. OLAP

- OLAP
  - Online Analytical Processing Database
  - Read-only, no updates, data is re-written in batches offline (e.g., once a day)
  - Usually larger queries (scans, aggregates)
  - Sometimes relational, sometimes not
  - Use cases
    - Interactive data analysis
    - Monthly business report
    - Design a statistical model
  - Examples: BigQuery, Snowflake, Databricks, Redshift, …

- We will focus in the next couple of weeks on **OLTP**

OLTP motivating example: an ATM



Read Balance
Give money
Update Balance

vs

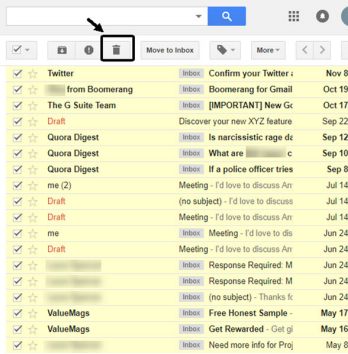
Read Balance
Update Balance
Give money

What if multiple applications/users are accessing the same table?

Visa does > 60,000 TXNs/sec with users & merchants

Want your 6$ Starbucks transaction to wait for a 10k$ bet in Las Vegas ?
(Transactions can (1) be quick or take a long time, (2) unrelated to you)

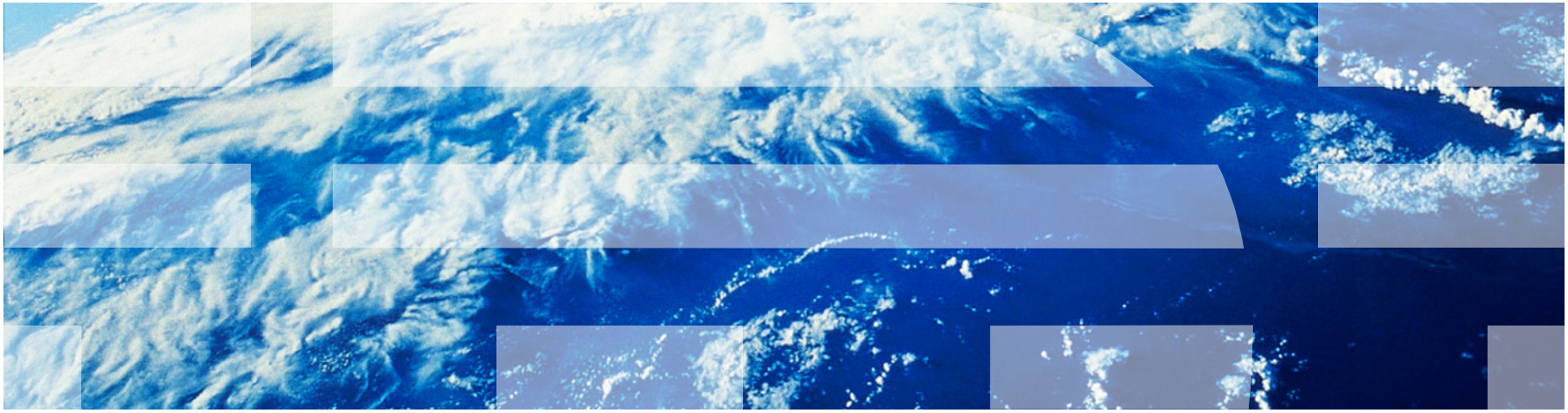
Transactions are not just used for finance



Transactions are at the core of

- -- payment, stock market, banks, ticketing
- -- Gmail, Google Docs (e.g., multiple people editing)
- -- Gaming
- -- Healthcare systems
- …

Transactions



Example: monthly bank interest transaction

Money

Account	....	Balance ($)
3001		500
4001		100
5001		20
6001		60
3002		80
4002		-200
5002		320
...		...
30108		-100
40008		100
50002		20

Money (@4:29 am day+1)

Account	....	Balance ($)
3001		550
4001		110
5001		22
6001		66
3002		88
4002		-220
5002		352
...		
30108		-110
40008		110
50002		22

'T-Monthly-423'

Monthly Interest 10%

4:28 am Starts run on 10M bank accounts

Takes 24 hours to run

```
UPDATE Money
SET Balance = Balance * 1.1
```

Example: monthly bank interest transaction **with crash**

Money

Account	....	Balance ($)
3001		500
4001		100
5001		20
6001		60
3002		80
4002		-200
5002		320
...		...
30108		-100
40008		100
50002		20

Money (@10:45 am)

Account	....	Balance ($)	
3001		550	??
4001		110	
5001		22	
6001		66	
3002		88	
4002		-200	
5002		320	??
...			
30108		-110	
40008		110	
50002		22	??

'T-Monthly-423'

Monthly Interest 10%
4:28 am Starts run on 10M bank accounts
Takes 24 hours to run
Network outage at 10:29 am,
System access at 10:45 am

Transactions: Basic Definition

A transaction ("TXN") is a sequence of one or more *operations* (reads or writes) which reflects *a single real-world transition*.

TXN either happened completely or not at all

```
START TRANSACTION
        UPDATE Product
        SET Price = Price – 1.99
        WHERE pname = 'Gizmo'
COMMIT
```

Transactions in SQL

- In “ad-hoc” SQL, each statement = one transaction

- In a program, multiple statements can be grouped together as a transaction

```
START TRANSACTION
        UPDATE Bank SET amount = amount – 100
        WHERE name = ‘Bob’
        UPDATE Bank SET amount = amount + 100
        WHERE name = ‘Joe’
COMMIT
```